Übungsblatt 5: Inter-Task-Kommunikation und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie wählen einen passenden Kommunikationsvertrag, rechnen Queue-Belegung und Zeitgrößen nach und untersuchen eine begrenzte Nachrichtenübergabe. Voraussetzung sind Vorlesungen 1–4. Die vier Papieraufgaben ergeben **30 Punkte** und sind ohne Board lösbar; ihre Rechenwege entsprechen möglichen Klausuraufgaben. Planen Sie 20 Minuten für Aufgabe 1, insgesamt 20 Minuten für Aufgaben 2–3 und 10 Minuten für Aufgabe 4.

Die Programmierphase dauert mit vorbereitetem Projekt etwa 35 Minuten: 10 Minuten Integration und 25 Minuten Versuch/Auswertung. Sie nutzt STM32CubeMX, VS Code mit STM32 VS Code Extension und das **NUCLEO-H755ZI-Q mit STM32H755ZIT6**. Eine neue Toolinstallation oder erstmalige Reparatur des Dual-Core-Starts ist Vorbereitung und gehört nicht in diese Zeit. Das Repository enthält das vollständige Anwendungsmodul in `code/lab05/`, kein generiertes Zielprojekt. Tickphase, Heap, Stack, GPIO, Interruptzuordnung und Prioritäten werden am bereitgestellten Projekt geprüft. Ein erfolgreicher Build beweist noch keine Echtzeitfrist.

Aufgabe 1: Ring und Wartezustände · 8 Punkte

Lehrkonvention: FIFO, $N = 3$, Head ist der nächste Entnahmeplatz, Tail der nächste Einfügeplatz, Count die Belegung. Alle drei Plätze sind nutzbar. Start: Head = Tail = Count = 0. Ein volles Try-Put verwirft den neuen Wert und ändert keine Queue-Daten. Ein leeres Try-Get liefert keine neue Nachricht. Try bedeutet Timeout null, ohne Warten.

1. Verfolgen Sie Put 10, Put 20, Get, Put 30 und Put 40. Geben Sie nach jedem Schritt logische Folge, Head, Tail und Count an. Geben Sie danach die physische Slotbelegung an. (3 P)
2. Setzen Sie fort: Put 50, Get, Put 50, Get, Get, Get, Get. Geben Sie je Operation Erfolg oder Fehler, einen gegebenenfalls gelieferten Wert und den neuen Zustand an. (3 P)
3. Welche zwei Belegungen können bei Head = Tail auftreten? Erklären Sie außerdem den Unterschied zwischen „Get wartet auf Daten“, „Put macht den Consumer Ready“ und „Consumer verarbeitet“. (2 P)

Musterlösung Aufgabe 1

Schritt	Ergebnis	Logische Folge	Head	Tail	Count
Start	–	leer	0	0	0
Put 10	Erfolg	10	0	1	1
Put 20	Erfolg	10, 20	0	2	2
Get	10	20	1	2	1
Put 30	Erfolg	20, 30	1	0	2
Put 40	Erfolg	20, 30, 40	1	1	3
Put 50	voll, Fehler	20, 30, 40	1	1	3
Get	20	30, 40	2	1	2
Put 50	Erfolg	30, 40, 50	2	2	3
Get	30	40, 50	0	2	2
Get	40	50	1	2	1
Get	50	leer	2	2	0
Get	leer, Fehler	leer	2	2	0

Nach dem ersten Put 40 liegen physisch `[40, 20, 30]` in den Slots. Die Folge beginnt bei Head, nicht zwingend bei Slot null. Gleiche Indizes können bei Count null leer und bei Count drei voll bedeuten. Alte Bytes in freigegebenen Slots sind keine gültigen Elemente.

Ein erlaubtes wartendes Get auf leerer Queue setzt den Consumer auf Blocked. Der erfolgreiche Put kann ihn Ready machen; erst die Auswahl durch den Scheduler lässt ihn Running werden und verarbeiten. Bei Timeout null blockiert der Get nicht. Ein abgelaufener oder fehlgeschlagener Get darf seinen Zielpuffer nicht als neue Nachricht verarbeiten.

Aufgabe 2: Überlauf und Kapazität · 8 Punkte

Lehrmodell, keine Messung: Queue anfangs leer, $N = 10$. Ein Put bei jeder ganzen Millisekunde von 1 bis 20 versucht den Wert seines Zeitpunkts einzureihen. Get bei 5, 10, 15 und 20 ms entnimmt je ein Element. Zeitgleich gilt **Put vor Get**. API-Zeit ist vernachlässigt. Volle Queue verwirft den neuen Datensatz.

1. Bestimmen Sie Belegung nach 5 und 10 ms und den ersten Fehl-Put. (2 P)
2. Bestimmen Sie verworfene Werte, entnommene Werte, Zahl erfolgreicher Puts und Restbelegung bei 20 ms. Prüfen Sie beide Nachrichtenbilanzen. (3 P)
3. Wie groß müsste die Queue sein, um nur dieses begrenzte Intervall verlustfrei zu speichern? Begründen Sie die höchste Zwischenbelegung. Kann eine endliche Vergrößerung bei dauerhaft denselben Raten jeden Verlust verhindern? (2 P)
4. Wählen Sie eine begründete Überlaufregel für einen Softwareauftrag, dessen Erzeugung warten darf, und für ein Display, das nur den neuesten Zustand braucht. (1 P)

Musterlösung Aufgabe 2

Nach 5 ms liegen vier, nach 10 ms acht Nachrichten vor. Bei 12 ms ist die Queue voll, der erste Fehl-Put liegt bei 13 ms. Verworfen werden 13, 14, 15, 17, 18, 19 und 20. Der Get bei 15 ms schafft erst nach dem dort abgelehnten Put einen Platz; deshalb gelingt Put 16. Die Gets liefern 1, 2, 3 und 4. Es gibt 13 erfolgreiche Puts, sieben Verluste und neun Restnachrichten.

Die Bilanzen lauten $20 = 13 + 7$ für Versuche, Erfolge und Verluste sowie $13 = 4 + 9$ für Erfolge, Entnahmen und Rest. Ein verlustfreier Referenzverlauf erreicht unmittelbar nach Put 20 und vor Get 20 die Belegung $20 - 3 = 17$. Daher sind 17 Plätze erforderlich, obwohl nach dem letzten Get nur 16 übrig wären.

Die angebotene Rate beträgt 1000 Nachrichten/s, die Entnahmerate 200 Nachrichten/s. Dauerhaft wächst der verlustfreie Rückstau um 800 Nachrichten/s. Ein endlicher Puffer verschiebt nur den Verlust. Der wartbare Softwareauftrag kann durch blockierendes Put Gegendruck erhalten, sofern kein benötigter Mutex gehalten wird. Das Display kann einen expliziten Latest-Value-Vertrag nutzen. Ein Try-Put auf einer vollen 1er-Queue lässt den alten Zustand stehen und erfüllt diesen Vertrag nicht.

Aufgabe 3: Kopie, Besitz und Pflichtempfänger · 6 Punkte

1. Eine Wertqueue kopiert einen Datensatz mit `sample = 10`. Der Sender ändert anschließend seine lokale Kopie auf 99. Was erhält der spätere Consumer? Warum gilt dieselbe Begründung nicht für den Inhalt hinter einem kopierten Pointer? (2 P)
2. Eine Queue bietet zehn reine Pointerplätze. Es existieren zwei Bildpuffer A und B mit je 75 KiB. A wird vom Consumer gelesen, B ist erfolgreich eingereiht. Wie viele neue unabhängige Bilder kann der Producer jetzt befüllen? Berechnen Sie Pointer-Slot-Nutzspeicher bei angenommenen 4-Byte-Pointern und den tatsächlich vorhandenen Bildspeicher. (2 P)
3. Ein Pointer-Put scheitert. Wer bleibt für den Puffer verantwortlich? Reicht ein `static`-Array als Schutz vor zu frühem Wiederbefüllen? Erhalten zwei Get-Consumer derselben Queue beide jede Nachricht? (2 P)

Musterlösung Aufgabe 3

Der spätere Consumer erhält zehn, sofern die gespeicherte Nachricht nicht anderweitig entnommen oder verworfen wurde. Die Queue hat die vereinbarten Bytes kopiert. Bei einem Pointer wird nur die Adresse kopiert; beide Teilnehmer beziehen sich auf dieselben externen Bytes. Deren Gültigkeit und unveränderte Verfügbarkeit brauchen einen eigenen Besitzvertrag.

Kein weiterer Bildpuffer ist frei. Zehn Pointerplätze ergeben $10 \cdot 4 = 40$ Byte Elementnutzspeicher. Zwei Bilder benötigen $2 \cdot 75 = 150$ KiB, also 153600 Byte. Kontrollblock, Alignment und Allokatorverwaltung kommen hinzu. Die reine Queue-Kapazität erzeugt keine zusätzlichen Bilder.

Nach fehlgeschlagenem Put bleibt der Producer verantwortlich. Er darf später erneut versuchen oder den Puffer genau einmal in den Free-Pool zurückgeben. `static` sichert Lebensdauer, aber keine exklusive Nutzung. Zwei Get-Consumer verteilen Nachrichten: Ein Element wird nur einmal entnommen. Jeder Pflichtempfänger braucht einen eigenen Zustellvertrag, beispielsweise eine eigene Queue samt Fehlerbehandlung.

Aufgabe 4: Interruptübergabe und Zeitbudget · 8 Punkte

1. Lehrannahmen: vier implementierte NVIC-Prioritätsbits, passende Präemptionsgruppierung, logische Kernelaufrufgrenze fünf. Darf IRQ-Priorität drei `osMessageQueuePut(q, &m, 0, 0)` aufrufen? Wie steht es um Priorität sieben? Darf `osMutexAcquire(mutex, 0)` bei Priorität sieben aufgerufen werden? Berechnen Sie den geschobenen Registerwert der Grenze fünf und nennen Sie den Wert für den HAL-Prioritätssetter. (3 P)
2. Synthetische Spur in µs: Ereignis 0, ISR-Eintritt 3, Worker Ready 8, ISR-Ende 12, Workerstart 16, Abschluss 24. Bestimmen Sie Eintrittslatenz, ISR-Dauer, Ready-bis-Running, Startlatenz und Antwortzeit bis Abschluss. Prüfen Sie eine relative Deadline von 25 µs. Warum darf Ready-bis-Running nicht zusätzlich auf alle anderen Intervalle addiert werden? (3 P)

3. Ein anfangs leerer binärer Semaphor mit Maximum eins erhält drei Releases vor jedem Acquire. Wie viele Releases gelingen? Unterscheiden Sie diese Zusammenfassung von Tasterprellen und einem Queue-Drop. (2 P)

Musterlösung Aufgabe 4

Priorität drei ist dringlicher als die Grenze fünf und darf keine Kernel-API rufen. Priorität sieben liegt im zulässigen Bereich; Queue Put ist mit gültigem Handle und Timeout null aus einer erlaubten ISR vorgesehen. Mutex Acquire bleibt auch dort unzulässig. Die drei Bedingungen heißen erlaubte Funktion, nicht wartender Parameter und passende IRQ-Priorität.

Der Registerwert ist $5 \cdot 2^{8-4} = 80 = 0x50$. Der HAL-Setter erhält die logische fünf, nicht 0x50. Diese Zahlen gelten nur unter den gegebenen Annahmen.

Eintrittslatenz: $3 - 0 = 3$ µs. ISR-Dauer: $12 - 3 = 9$ µs. Ready-bis-Running: $16 - 8 = 8$ µs. Startlatenz: $16 - 0 = 16$ µs. Antwortzeit bis Abschluss: $24 - 0 = 24$ µs. Die Frist 25 µs wird mit 1 µs Restbudget erfüllt. Ready-bis-Running überlappt einen Teil der ISR und den Übergang. Alternativ zerlegt man ohne Überlappung $3 + 9 + 4 + 8 = 24$ µs. Der letzte Summand ist Workerarbeit, nicht zusätzliches Ready-Warten.

Ein Release gelingt, zwei melden fehlende Kapazität. Das ist Zusammenfassung mehrerer Arbeitshinweise. Prellen erzeugt mehrere elektrische Flanken; ein Queue-Drop lehnt eine bereits erfasste Nachricht wegen Vollbelegung ab. Hardware-Pendingbits können schon vor dem Callback zusammenfassen. Keine dieser Zählungen ist automatisch die Zahl entprellter Betätigungen.

Aufgabe 5: Wertqueue im vorhandenen CM4-Projekt

Vorbereitung durch die Lehrperson

Verwenden Sie die lauffähige Basis aus Vorlesung 3, mit einem FreeRTOS-Kernel auf CM4 und dem vorhandenen CM7-Systemstart. In STM32CubeMX prüfen Sie CMSIS_V2, Präemption, dynamische Queue-/Semaphorunterstützung und einen 32-Bit-Kernel-Tick mit 1000 Hz. Übernehmen Sie die begründete HAL-Zeitbasis der Basis. Deaktivieren Sie die bisherigen Lastthreads, Semaphore-/Mutexversuche und Buttoncallbacks. Es laufen in Teil A nur die beiden heutigen Anwendungsthreads zusätzlich zu den Kernelthreads.

Erzeugen Sie Producer mit `osPriorityNormal` und Consumer mit `osPriorityAboveNormal`, zunächst je 1024 Byte Stack. Prüfen Sie Heap und Stack am konkreten Build und unter Laufzeitlast. STM32CubeMX erzeugt die Quellen; Build, Flash und Debug erfolgen in VS Code mit STM32 VS Code Extension für die zugehörigen CM4- und CM7-CMake-Ziele. Es wird kein Wechsel zur eigenständigen STM32CubeIDE verlangt.

Integration und Durchführung

1. Kopieren Sie `code/lab05/lab05_messages.c` und `.h` in den CM4-Anwendungsbereich. Fügen Sie die `.c`-Datei dem tatsächlichen CM4-CMake-Ziel und ihren Ordner den Include-Verzeichnissen hinzu. Nutzen Sie eine von Cube-Regeneration erhaltene Anpassungsstelle. Ein bloßer Dateikopiervorgang garantiert keine Einbindung in den Build.
2. Fügen Sie `#include "lab05_messages.h"` im vorgesehenen Benutzerbereich ein. Rufen Sie `Lab05_Init()` **einmal nach dem vorhandenen erfolgreichen** `osKernelInitialize()` **und vor der Threadanlage** auf. Der Kernelstart wird nicht dupliziert. Das Modul prüft alle Handles; ein Fehler stoppt die Anwendung.

3. Die generierte Producer-Einstiegsfunktion ruft nur `Lab05_Producer(argument)` auf, die Consumer-Einstiegsfunktion nur `Lab05_Consumer(argument)`. Legen Sie keine zweiten gleichnamigen Cube-Einstiegsfunktionen an. Der Button-Interrupt bleibt deaktiviert.
4. Bauen und flashen Sie das vorbereitete Dual-Core-Projekt. Starten Sie den Versuch nach Reset. Die Datenqueue besitzt vier Plätze. Der Producer bietet Sequenzen 1–20 mit `sample = 10 * seq` an, verwendet Timeout null und wartet danach jeweils zehn Ticks. Der Consumer entnimmt mit `osWaitForever`, protokolliert und wartet je Datenmessage vierzig Ticks. Beide Wartezeiten sind **keine exklusive CPU-Last** und kein exakt garantiertes periodisches Raster.
5. Nach den zwanzig Versuchen sendet der Producer eine STOP-Nachricht mit wartendem Put. Die STOP-Nachricht hat einen eigenen Typ und zählt nicht zu den Datenversuchen. Der aktive Consumer leert vorherige Nachrichten und parkt nach STOP. Der Producer parkt nach erfolgreicher Übergabe. Niemand released den leeren Park-Semaphor.
6. Halten Sie alle Kerne nach Abschluss an und prüfen Sie `lab05_producer_done`, `lab05_consumer_done`, `lab05_attempt`, `lab05_received`, `lab05_accepted`, `lab05_dropped` und `lab05_received_count`. Interpretieren Sie die Bilanz erst bei beiden Done-Werten eins. Andere als erwartete Kapazitätsfehler führen zum Fehlerpfad und damit zu keinem gültigen abgeschlossenen Versuch.

Prüfkriterium	Eigene Beobachtung
Tickrate und Threadprioritäten	
Build/Flash/Start tatsächlich erfolgreich	
Beide Done-Werte nach STOP	
20 Datenversuche = angenommen + verworfen	
Nach STOP: angenommen = empfangen	
Streng steigende empfangene Sequenzen	
Inhalt jeder Nachricht: 10 × Sequenz	
Welche Sequenzen fehlen laut Producerlog?	

Ändern Sie für einen zweiten Lauf die Consumerpause im Modul von vierzig auf zehn Ticks, bauen und resetten Sie erneut. Halten Sie Queuegröße und andere Einstellungen gleich. Begründen Sie die beobachtete Veränderung. Gleiche mittlere Wartewerte garantieren wegen tatsächlicher Ausführungszeiten und Tickphase keine beliebige verlustfreie Dauerlast. Halten Sie die beobachteten Zahlen fest, statt die Papierwerte aus Aufgabe 2 einzutragen.

Musterlösung Aufgabe 5

Die vollständigen Threads liegen in `code/lab05/lab05_messages.c`. Es werden keine zusätzlichen Schleifen um ihre Aufrufe gebaut; beide kehren im gültigen Ablauf nicht zurück. Bei Teil A schreiben Producer und Consumer ausschließlich ihre eigenen Diagnosebereiche. Der normale Daten-Put akzeptiert oder verwirft die neue Nachricht. Andere Fehler stoppen den Versuch. Get wird nur nach `osOK` ausgewertet. Die private Wertkopie schützt die empfangenen Zahlen vor Wiederverwendung der Producer-Variable.

Die langsamere Consumerpause erzeugt erwartbar Rückstau und Kapazitätsverluste, deren genaue Anzahl von Tickphase, Scheduling und Build abhängt. Nach STOP und angehaltenem Ablauf gilt 20 = accepted + dropped sowie accepted = received. STOP steht hinter allen angenommenen Daten und wird separat verarbeitet. Fehlende Endsequenzen sieht man im Producerlog, auch wenn das Consumerlog keinen späteren Wert mehr enthält.

Eine kürzere Consumerpause reduziert den Rückstau unter sonst gleichen Bedingungen. Ohne Boardlauf werden keine Verlustzahl und kein erfolgreiches Flashen behauptet. Tickstempel

liefern grobe verstrichene Zeit, keinen Nachweis exklusiver CPU-Ausführungszeit oder einer Mikrosekundenfrist.

Aufgabe 6: Buttonereignis aus EXTI an einen Worker

Teil B wird **nach Reset als eigener Versuch** durchgeführt. Deaktivieren Sie Producer und Consumer. Ein ButtonWorker mit `osPriorityAboveNormal` und zunächst 1024 Byte Stack bleibt. `Lab05_Init()` wird weiterhin einmal vor Threadanlage ausgeführt.

1. Prüfen Sie Boardrevision, Lötbrücken, Pin, Aktivpegel und Flanke anhand der bereitgestellten Unterlagen. Bei bestätigtem PC13 wird dessen EXTI-Leitung dem CM4 zugeordnet. Nutzen Sie in anderen bestätigten Konfigurationen die passende Pinmaske und IRQ. Deaktivieren Sie einen unkoordinierten CM7-Handler derselben Leitung.
2. Prüfen Sie `__NVIC_PRIO_BITS`, die Gruppierung und die Definitionen `configLIBRARY_MAX_SYSCALL_INTER` sowie `configMAX_SYSCALL_INTERRUPT_PRIORITY`. Setzen Sie die **logische** EXTI-Priorität auf einen im konkreten Projekt zulässigen Wert. Es gilt nicht allgemein „immer fünf“.
3. Halten Sie diese NVIC-IRQ während GPIO-Initialisierung, Objektanlage und Kernelstart deaktiviert. Falls CubeMX einen frühen Enable-Aufruf generiert, passen Sie die Generierungs-/Benutzerstelle so an, dass kein Zeitfenster für einen frühen Callback bleibt. Die Freigabe erfolgt erst am Anfang der laufenden Button-Einstiegsfunktion, vor `Lab05_ButtonWorker(argument)`.
4. Bei bestätigtem PC13 kann der folgende Code im laufenden Thread verwendet werden. Er verwirft absichtlich vor der Freigabe anstehende Altflags. Konfiguration und Freigabe bilden eine dokumentierte Erfassungsgrenze; ein Ereignis exakt an dieser Grenze ist nicht als sicher erfasst behauptet. Prüfen Sie die Makros und den Handler im lokalen HAL-Paket.

```
/* Nur für bestätigte EXTI13/EXTI15_10_IRQn-Konfiguration auf CM4. */
__HAL_GPIO_EXTI_CLEAR_IT(GPIO_PIN_13);
HAL_NVIC_ClearPendingIRQ(EXTI15_10_IRQn);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
Lab05_ButtonWorker(argument);
```

5. Der generierte CM4-IRQ-Handler ruft `HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13)` auf. HAL behandelt das Hardwareflag und ruft den Callback. Fügen Sie den Callback genau einmal außerhalb generierter Funktionstexte hinzu, beispielsweise in einer erhaltenen Benutzer-Code-Sektion. Keine zweite Quittung und kein zusätzliches Yield.

```
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == GPIO_PIN_13) {
        Lab05_ButtonIRQ();
    }
}
```

6. Das Modul übergibt Sequenz und Kernel-Tick in eine separate Queue mit vier Plätzen. Der Worker liest sie und pausiert fünfzig Ticks. Drücken Sie den Taster zunächst langsam, dann mehrfach. Beobachten Sie nach angehaltenen Kernen `lab05_button_offered`, `lab05_button_accepted`, `lab05_button_dropped`, `lab05_button_errors`, `lab05_button_processed` und `lab05_button_last`. Die Zähler werden nicht in einem parallelen Reportingthread gelesen oder zurückgesetzt. Bei laufender Anwendung ist eine zusammengehörige Momentaufnahme nicht garantiert.

7. Unterscheiden Sie Callbackanzahl, empfangene Nachrichten und echte Betätigungen. Das Modul enthält **keine Entprellung**. Mechanisches Prellen muss nicht in jedem Lauf einen Überlauf erzeugen; ein unbeobachteter Drop wird nicht erfunden. Eine Sequenzlücke zeigt nicht die Zahl vor dem Callback zusammengefasster Flanken.
8. Optional: Beobachten Sie ISR-Eintritt/-Austritt und Workerstart über zusätzliche GPIO-Marker mit Logic Analyzer oder geeignetem Trace. Marker müssen am richtigen Punkt im tatsächlichen IRQ-/Workerpfad sitzen. Dokumentieren Sie Zeitbasis und Messkosten. Ohne Messgerät lassen Sie ISR-Dauer, Workerstartlatenz und Mikrosekundenwerte offen. UART-Ausgabe bleibt außerhalb der ISR und außerhalb der bewerteten Zeitabschnitte.

Aussage	Beobachtung und Begründung
Erlaubte Funktion, Timeout, IRQ-Priorität	
Offered = Accepted + Dropped + technische Put-Fehler?	
Noch wartende Nachrichten bzw. vollständig geleert?	
Callbacksequenz gegenüber Betätigungen	
Grenzen der Tickstempel und Diagnose	

Die zweite Bilanz gilt nur bei gültigem Start und ohne Diagnosezählerüberlauf. `lab05_button_errors` enthält auch erkannte Frühaufrufe, die nicht als angeboten gezählt wurden. Ein solcher Frühaufruf ist ein Konfigurationsfehler und wird vor der Bilanz korrigiert. Alle ISR-Zähler haben genau einen Schreiber, die eine geprüfte CM4-Button-IRQ; `volatile` ersetzt keine allgemeine Synchronisation.

Musterlösung Aufgabe 6

Im erlaubten HAL-Pfad genügt der Callback mit `Lab05_ButtonIRQ()`. Der Adapter erledigt bei erfolgreichem Put die erforderliche native Wechselanforderung. Der Worker wird Ready und kann erst nach ISR-Ende und zulässigem Scheduling Running werden. Seine Threadpriorität macht eine zu dringliche NVIC-Priorität nicht zulässig.

Die Queue bewahrt nur angenommene Callbacknachrichten und ist endlich. Bei vollem Kanal zählt der neue Callback als Drop, ältere Nachrichten bleiben stehen. `lab05_button_processed` wird beim erfolgreichen Get erhöht; die anschließende Pause ist nicht Teil einer behaupteten Entprellung. Bei gesperrter Quelle und vollständig entnommener Queue entspricht angenommen der Zahl entnommener Nachrichten. Bei angehaltenem Ablauf mit Rückstau muss dieser Rest berücksichtigt werden. Eine kurze Debuggeranzeige einer LED oder ein erfolgreicher Build ersetzt keine Messung der fertigen Reaktion.

Ein gültiger Start hat keine Frühaufrufe und keine technischen Fehler. Das Diagnosefeld für technische Fehler dient dem Erkennen eines ungültigen Laufs. Es wird in der ISR kein potentiell blockierender Error_Handler aufgerufen. Für die reine Boardbeobachtung bleiben konkrete Messwerte bis zum tatsächlichen Lauf leer.